

Programare Orientată Pe Obiecte

Curs 9

Colectii și generice în Java

Quizz

Noiembrie 2025

Quizz Colectii

Întrebarea 1 – List pentru coș de cumpărături

```

1 import java.util.ArrayList;
2 import java.util.List;
3
4 public class CosCumparaturi {
5     public static void main(String[] args) {
6         ArrayList<String> produse = new ArrayList<>();
7         produse.add("Lapte");
8         produse.add("Paine");
9         produse.add("Oua");
10        produse.add(1, "Cafea");
11
12        System.out.println(produse);
13    }
14 }
  
```

Ce se afișează?

- A) [Lapte, Paine, Oua, Cafea]
- B) [Lapte, Cafea, Paine, Oua]
- C) [Cafea, Lapte, Paine, Oua]
- D) Eroare la compilare

Întrebarea 1 – Răspuns

- A) [Lapte, Paine, Oua, Cafea]
B) **[Lapte, Cafea, Paine, Oua]**
C) [Cafea, Lapte, Paine, Oua]
D) Eroare la compilare

Explicație:

Metoda `add(int index, E element)` inserează elementul pe poziția 1 (a doua poziție):

- 1 După primele 3 add: [Lapte, Paine, Oua]
- 2 Apoi add(1, "Cafea") inserează: [Lapte, Cafea, Paine, Oua]

Întrebarea 2 – List de comenzi

```
1 import java.util.*;
2
3 public class Comenzi {
4     public static void main(String[] args) {
5         List<String> comenzi = new ArrayList<>();
6         comenzi.add("Pizza");
7         comenzi.add("Burger");
8         comenzi.add("Salata");
9         comenzi.remove("Burger");
10        comenzi.remove(0);
11
12        System.out.println(comenzi);
13    }
14 }
```

Ce rămâne în listă?

- A) [Pizza, Salata]
B) [Burger, Salata]
C) [Salata]
D) Lista este goală

Întrebarea 4 – Map pentru stoc de produse

```

1 import java.util.*;
2
3 public class StocMagazin {
4     public static void main(String[] args) {
5         Map<String, Integer> stoc = new HashMap<>();
6         stoc.put("Lapte", 10);
7         stoc.put("Paine", 5);
8         stoc.put("Lapte", 15);
9
10        System.out.println(stoc.get("Lapte"));
11    }
12 }

```

Ce valoare se afișează pentru "Lapte"?

- A) 10
- B) 5
- C) 15
- D) null

Întrebarea 4 – Răspuns

- A) 10
- B) 5
- C) **15**
- D) null

Explicație:

Map.put cu aceeași cheie rescrie valoarea anterioară:

- put("Lapte", 10) pune 10
- put("Lapte", 15) rescrie valoarea 10 cu 15

Întrebarea 5 – Iterare prin Map (catalog note)

```

1 import java.util.*;
2
3 public class Catalog {
4     public static void main(String[] args) {
5         Map<String, Integer> note = new HashMap<>();
6         note.put("Ana", 10);
7         note.put("Ion", 8);
8         note.put("Maria", 9);
9
10        for (String nume : note.keySet()) {
11            System.out.println(nume + " -> " + note.get(nume));
12        }
13    }
14 }

```

Ce descrie cel mai bine codul?

- A) Afișează doar cheile
- B) Afișează doar valorile
- C) Afișează perechile nume-notă
- D) Produce eroare la compilare

Întrebarea 5 – Răspuns

- A) Afișează doar cheile
- B) Afișează doar valorile
- C) **Afișează perechile nume-notă**
- D) Produce eroare la compilare

Explicație:

Se parcurge `keySet()`, iar pentru fiecare cheie `nume` se afișează și valoarea corespunzătoare `note.get(nume)`.

Întrebarea 6 – Queue pentru coadă de clienți

```

1 import java.util.*;
2
3 public class CoadacClienti {
4     public static void main(String[] args) {
5         Queue<String> coada = new LinkedList<>();
6         coada.add("Ana");
7         coada.add("Ion");
8         coada.add("Maria");
9
10        System.out.println(coada.poll());
11        System.out.println(coada.peek());
12    }
13 }

```

Ce se afișează?

- A) Ana apoi Ion
- B) Ana apoi Maria
- C) Ion apoi Maria
- D) Ion apoi Ana

Știm că:

- `poll()` returnează și elimină primul element
- `peek()` returnează primul element fără să-l elimine

Întrebarea 6 – Răspuns

- A) **Ana apoi Ion**
- B) Ana apoi Maria
- C) Ion apoi Maria
- D) Ion apoi Ana

Explicație:

- `poll()` returnează și elimină primul element: "Ana"
- după `poll()`, coada devine ["Ion", "Maria"]
- `peek()` returnează primul element fără să-l elimine: "Ion"

Întrebarea 7 – TreeSet pentru ordonare

```

1 import java.util.*;
2
3 public class OrdonareNume {
4     public static void main(String[] args) {
5         Set<String> nume = new TreeSet<>();
6         nume.add("Mihai");
7         nume.add("Ana");
8         nume.add("Ion");
9
10        System.out.println(nume);
11    }
12 }

```

Ce proprietate are colecția nume?

- A) Elementele sunt în ordinea inserării
- B) Elementele sunt în ordine alfabetică
- C) Elementele sunt în ordine aleatoare
- D) Permite duplicate

Întrebarea 7 – Răspuns

- A) Elementele sunt în ordinea inserării
- B) **Elementele sunt în ordine alfabetică**
- C) Elementele sunt în ordine aleatoare
- D) Permite duplicate

Explicație:

`TreeSet` sortează elementele conform ordinii naturale (pentru `String` → ordine alfabetică) și nu permite duplicate.

Quizz Generice

Întrebarea 1 – Tip generic simplu

```

1 class Cutie<T> {
2     private T continut;
3     public void set(T continut) {
4         this.continut = continut;
5     }
6     public T get() {
7         return continut;
8     }
9 }
10 public class Test {
11     public static void main(String[] args) {
12         Cutie<Integer> c = new Cutie<>();
13         c.set(10);
14         Integer x = c.get();
15         System.out.println(x);
16     }}

```

Ce face programul?

- A) Afişează 10
- B) Generează eroare la compilare
- C) Generează eroare la rulare
- D) Afişează null

Întrebarea 1 – Răspuns

- A) **Afișează 10**
- B) Generează eroare la compilare
- C) Generează eroare la rulare
- D) Afișează null

Explicație: Obiectul `Cutie<Integer>` reține valoarea 10. Metoda `get()` o returnează, iar `System.out.println(x)` afișează 10.

Întrebarea 2

```

1 class Cutie<T> {
2     T valoare;
3 }
4
5 public class Test {
6     public static void main(String[] args) {
7         Cutie c = new Cutie();           // raw type
8         c.valoare = "Salut";
9
10        Integer x = (Integer) c.valoare;
11        System.out.println(x);
12    }
13 }

```

Ce se întâmplă?

- A) Codul nu compilează
- B) Compilează și afișează Salut
- C) Compilează, dar aruncă o Exceptie la rulare
- D) Afișează null

Întrebarea 2 – Răspuns

- A) Codul nu compilează
- B) Compilează și afișează Salut
- C) **Compilează, dar aruncă o Exceptie (ClassCastException) la rulare**
- D) Afișează null

Explicație: Se folosește tipul brut `Cutie`, iar în câmpul `valoare` se pune un `String`. La rulare se face cast la `Integer`, ceea ce provoacă `ClassCastException`.

Întrebarea 3 – Generic bounds

```

1 class Calculator<T extends Number> {
2     private T valoare;
3
4     public Calculator(T valoare) {
5         this.valoare = valoare;
6     }
7
8     public double dublu() {
9         return valoare.doubleValue() * 2;
10    }
11 }

```

Care dintre următoarele instanțieri sunt valide?

- A) `Calculator<Integer> c = new Calculator<>(10);`
- B) `Calculator<Double> c = new Calculator<>(2.5);`
- C) `Calculator<String> c = new Calculator<>("abc");`
- D) Toate de mai sus

Întrebarea 3 – Răspuns

- A) `Calculator<Integer> c = new Calculator<>(10);`
- B) `Calculator<Double> c = new Calculator<>(2.5);`
- C) `Calculator<String> c = new Calculator<>("abc");`
- D) Toate de mai sus

Răspuns corect: A și B

Explicație: T este constrâns cu `extends Number`. `Integer` și `Double` sunt subclase ale `Number`; `String` nu respectă această constrângere.

Întrebarea 4 – Metodă generică

```

1 public class Util {
2     public static <T> void afiseaza(T element) {
3         System.out.println(element);
4     }
5
6     public static void main(String[] args) {
7         afiseaza(100);
8         afiseaza("Java");
9         afiseaza(3.14);
10    }
11 }

```

Ce se afișează?

- A) 100 Java 3.14
- B) Eroare de compilare
- C) Doar 100
- D) null null null

Întrebarea 4 – Răspuns

- A) **100 Java 3.14**
- B) Eroare de compilare
- C) Doar 100
- D) null null null

Explicație: Metoda generică `afiseaza` acceptă orice tip. Se apelează cu `Integer`, `String` și `Double`, iar fiecare este afișat pe linie separată.

Întrebarea 5 – Wildcards

```

1 import java.util.*;
2
3 public class Test {
4     public static void afiseazaLista(List<? extends Number> lista) {
5         for (Number n : lista) {
6             System.out.println(n);
7         }
8     }
9     public static void main(String[] args) {
10         List<Integer> nr = new ArrayList<>();
11         nr.add(1);
12         nr.add(2);
13         afiseazaLista(nr);
14     }
15 }

```

Ce este adevărat?

- A) Codul nu compilează
- B) Va afișa 1 și 2
- C) Va afișa null
- D) Va apărea ClassCastException

Întrebarea 5 – Răspuns

- A) Codul nu compilează
- B) **Va afișa 1 și 2**
- C) Va afișa null
- D) Va apărea ClassCastException

Explicație: `List<Integer>` este compatibil cu `List<? extends Number>`. Elemente sunt iterate și afișate ca `Number`, fără probleme.

Întrebarea 6 – Comparable vs Generic

```

1 class Produs implements Comparable<Produs> {
2     double pret;
3
4     public Produs(double pret) {
5         this.pret = pret;
6     }
7
8     @Override
9     public int compareTo(Produs p) {
10         return Double.compare(this.pret, p.pret);
11     }
12 }

```

Ce permite această implementare?

- A) Sortarea unei liste de Produs după pret
- B) Compararea cu orice tip de obiect
- C) Conversia Produs în double
- D) Dezactivarea tipului generic

Întrebarea 6 – Răspuns

- A) **Sortarea unei liste de `Produs` după preț**
- B) Compararea cu orice tip de obiect
- C) Conversia `Produs` în `double`
- D) Dezactivarea tipului generic

Explicație: Implementarea lui `Comparable<Produs>` permite folosirea metodelor de sortare (ex. `Collections.sort`) pentru liste de `Produs`, ordonate după preț.

Întrebarea 7 – Generic + moștenire

```
1 class A {}
2 class B extends A {}
3
4 List<A> listaA = new ArrayList<B>();
```

Ce se întâmplă?

- A) Compilează
- B) Eroare la compilare
- C) Eroare la rulare
- D) Inițializare implicită ca `List<Object>`

Întrebarea 7 – Răspuns

- A) Compilează
- B) **Eroare la compilare**
- C) Eroare la rulare
- D) Inițializare implicită ca `List<Object>`

Explicație: Deși B extinde A, `List<B>` nu este subtip de `List<A>`. Astfel, `new ArrayList<B>()` nu poate fi atribuit unei variabile de tip `List<A>`.

Generics în Java sunt invariante:

X extends Y nu implică `List<X>` extends `List<Y>`

Întrebarea 8 – Generic + moștenire

```
1 class A {}
2 class B extends A {}
3
4 List<B> listaB = new ArrayList<A>();
```

Ce se întâmplă?

- A) Compilează
- B) Eroare la compilare
- C) Eroare la rulare
- D) Inițializare implicită ca `List<Object>`

Întrebarea 8 – Răspuns

- A) Compilează
- B) **Eroare la compilare**
- C) Eroare la rulare
- D) Inițializare implicită ca `List<Object>`

Explicație: Deși B extinde A, `List<A>` nu este subtip de `List<B>`. Astfel, `new ArrayList<A>()` nu poate fi atribuit unei variabile de tip `List<B>`.

Generics în Java sunt invariante:

X extends Y nu implică `List<X>` extends `List<Y>`

Întrebarea 9

```

1 class Box<T> {
2     T value;
3 }
4
5 public class Test {
6     public static void main(String[] args) {
7         Box<Integer> box = new Box<>();
8         box.value = 5;
9     }
10 }

```

Care afirmație este adevărată?

- A) value este de tip Object
- B) value este de tip Integer
- C) Codul nu compilează
- D) value este de tip String

Întrebarea 9 – Răspuns

```
1 class Box<T> {
2     T value;
3 }
4
5 Box<Integer> box = new Box<>();
6 box.value = 5;
```

Care afirmație este adevărată?

- A) value este de tip Object
- B) value este de tip Integer**
- C) Codul nu compilează
- D) value este de tip String

Explicație: Tipul generic T este instanțiat cu Integer, deci câmpul value devine de tip Integer.

Întrebarea 10

```
1 class Suma<T extends Number> {
2     T valoare;
3 }
```

Care dintre următoarele instanțieri sunt valide?

- A) `Suma<Integer> s = new Suma<>();`
- B) `Suma<Double> s = new Suma<>();`
- C) `Suma<String> s = new Suma<>();`
- D) A și B

Întrebarea 10 – Răspuns

```
1 class Suma<T extends Number> {
2     T valoare;
3 }
```

Care dintre următoarele instanțieri sunt valide?

- A) Suma<Integer> s = new Suma<>();
- B) Suma<Double> s = new Suma<>();
- C) Suma<String> s = new Suma<>();
- D) **A și B**

Explicație: T este constrâns să extindă Number. Integer și Double sunt subclase ale Number, dar String nu este.

Întrebarea 11

```

1 import java.util.*;
2
3 public class Test {
4     public static void main(String[] args) {
5         List<String> lista = new ArrayList<>();
6         List<Object> lista2 = lista;
7     }
8 }

```

Ce se întâmplă?

- A) Compilează
- B) Eroare la compilare
- C) Eroare la rulare
- D) Lista devine List<Object>

Întrebarea 11 – Răspuns

```
1 List<String> lista = new ArrayList<>();
2 List<Object> lista2 = lista;
```

Ce se întâmplă?

- A) Compilează
- B) **Eroare la compilare**
- C) Eroare la rulare
- D) Lista devine List<Object>

Explicație: Generics NU sunt covariante în Java. List<String> nu este subtip de List<Object>, deci atribuirea este interzisă de compilator.

Întrebarea 12

```
1 import java.util.*;
2
3 public class Test {
4     public static void metoda(List<? super Integer> lista) {
5         lista.add(10);
6     }
7 }
```

Care apel este valid?

- A) metoda(new ArrayList<Integer>());
- B) metoda(new ArrayList<Number>());
- C) metoda(new ArrayList<Object>());
- D) Toate de mai sus

Întrebarea 12 – Răspuns

```
1 public static void metoda(List<? super Integer> lista) {
2     lista.add(10);
3 }
```

Care apel este valid?

- A) metoda(new ArrayList<Integer>());
- B) metoda(new ArrayList<Number>());
- C) metoda(new ArrayList<Object>());
- D) **Toate de mai sus**

Explicație: ? super Integer înseamnă "Integer sau orice superclasă a lui Integer". Sunt acceptate tipurile List<Integer>, List<Number> și List<Object>.

Întrebarea 13

```

1 class Pair<T, U> {
2     T first;
3     U second;
4 }
5
6 public class Test {
7     public static void main(String[] args) {
8         Pair<Integer, String> p = new Pair<>();
9     }
10 }

```

Care este tipul câmpului second?

- A) Integer
- B) Object
- C) String
- D) T

Întrebarea 13 – Răspuns

```
1 Pair<Integer, String> p = new Pair<>();
```

Care este tipul câmpului `second`?

- A) Integer
- B) Object
- C) **String**
- D) T

Explicație: Tipul generic `U` este instanțiat cu `String`, deci `second` are tipul `String`.

Întrebarea 14

```

1 import java.util.*;
2
3 public class Util {
4     public static <T> void copiaza(List<T> sursa, List<T> destinatie)
5     {
6         for (T elem : sursa) {
7             destinatie.add(elem);
8         }
9     }

```

Care apel este corect?

- A) copiaza(List<Integer>, List<Number>)
- B) copiaza(List<Number>, List<Integer>)
- C) copiaza(List<Integer>, List<Integer>)
- D) Toate sunt valide

Întrebarea 14 – Răspuns

```
1 public static <T> void copiaza(List<T> sursa, List<T> destinatie) {
2     for (T elem : sursa) {
3         destinatie.add(elem);
4     }
5 }
```

Care apel este corect?

- A) copiaza(List<Integer>, List<Number>)
- B) copiaza(List<Number>, List<Integer>)
- C) **copiaza(List<Integer>, List<Integer>)**
- D) Toate sunt valide

Explicație: Ambele liste trebuie să aibă același tip T. Doar List<Integer> cu List<Integer> respectă această condiție.